Software Engineering

CE2006/CZ2006

Requirements Analysis: Dynamic Models

Discussion Topics

- Interaction Diagrams
  - State Machine Diagrams
  - Activity Diagrams
- Software Requirement Specifications

- Reading
  - Bruegge chap. 2.4.3, 2.4.4, 2.4.5
  - Fowler chaps. 4, 10 and 12

State Machine Diagram

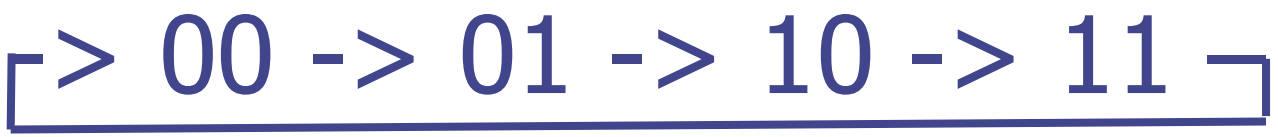
Also known as **Statecharts**, **Finite State Automata**.

Models **a system** or **an object** based on the "**States**" it can be in.

A "**State**" is a stable condition of a system, that can exist for a period of time, or for all time, until some **ACTION** drives the system into another stable state.

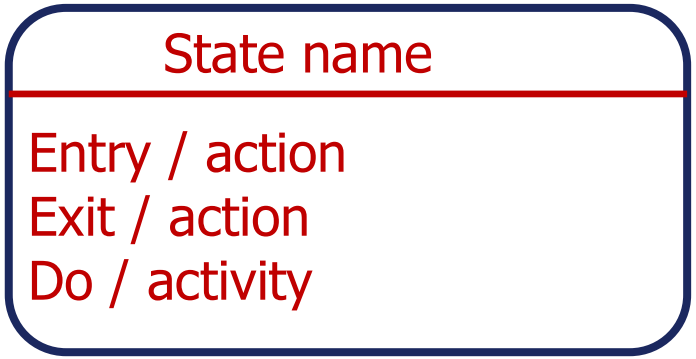
Most systems will have a **finite number of states** in which it can be depending on the previous state and actions which move it to another state.

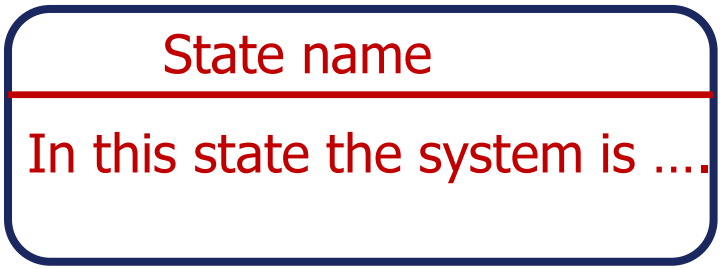
- e.g. a 2-bit binary counter has 4 states and moves from one state to the next when a clock signal drives it to the next state.



State Machine Diagram

In UML, a State is represented as a **rectangle** with **rounded** corners with a **name** at the top and **optionally** a **description** of the state and/or **actions and activities** performed while in that state.





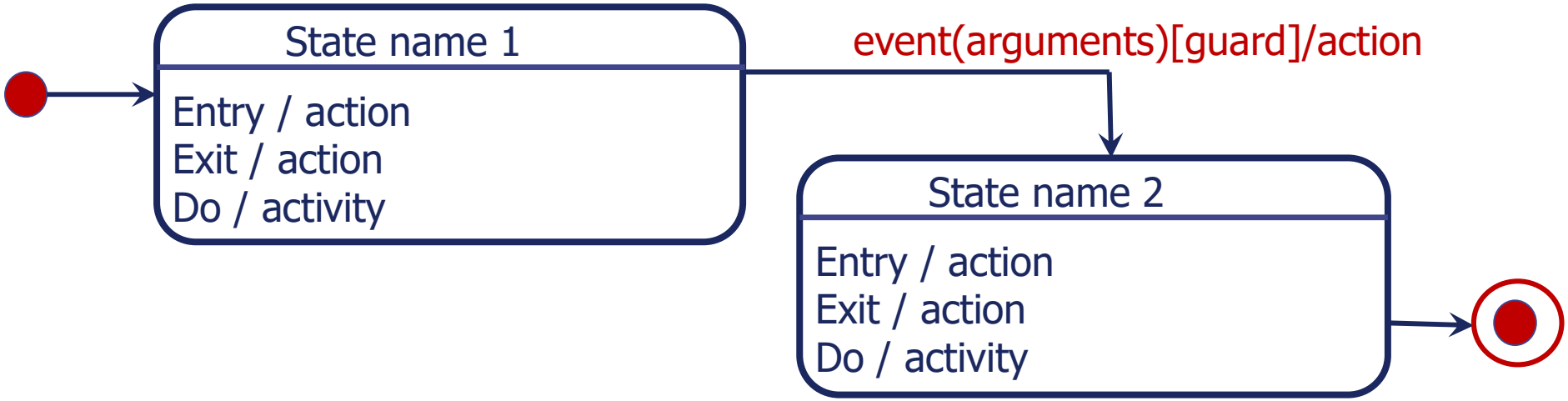


State Machine Diagram

A state machine diagram begins with a **start node** followed by a series of states connected by **transitions** and finally an **end node** (unless the system is in an infinite loop and there is no end node).

A transition from one state to the next is assumed to be instantaneous in response to an event. A transition can have **optionally**:

- event: event triggers the transition
- guard: the condition which must be true to take the transition
- action: action to do during the transition



Dialog Map: State Machine Diagrams in Prototyping

Prototyping is a technique to reduce the risk of building the wrong product, or of building the right product badly.

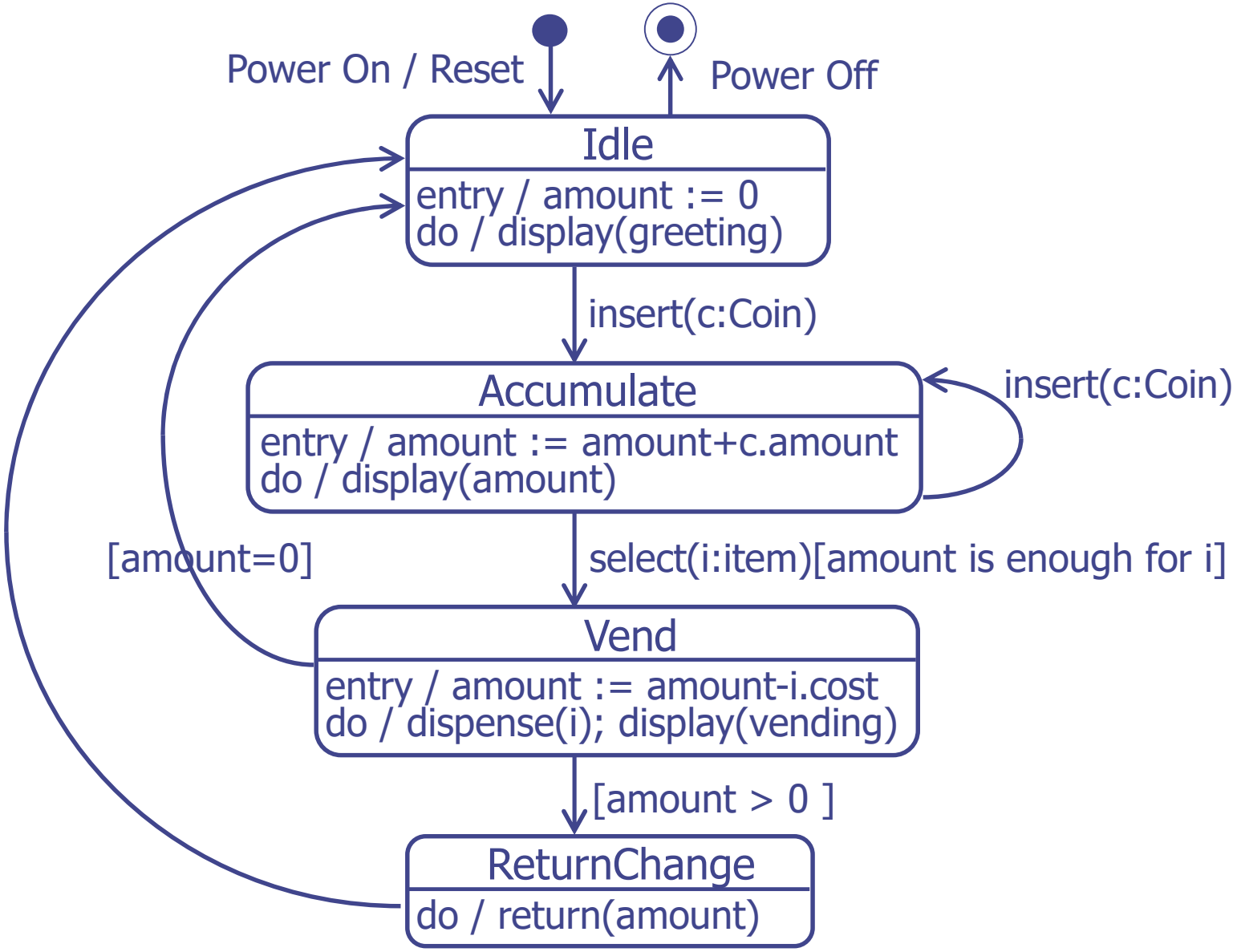
Using a State Machine Diagram and a prototype users can **experiment** with the **navigation options** and **preview** the **functionality** available at each screen to assess the look and feel of the system.

Many **user interfaces** can be thought of as **finite state machines**, with only one display screen or window active at a time (a state), and defined conditions for moving from one state to another (a transition).

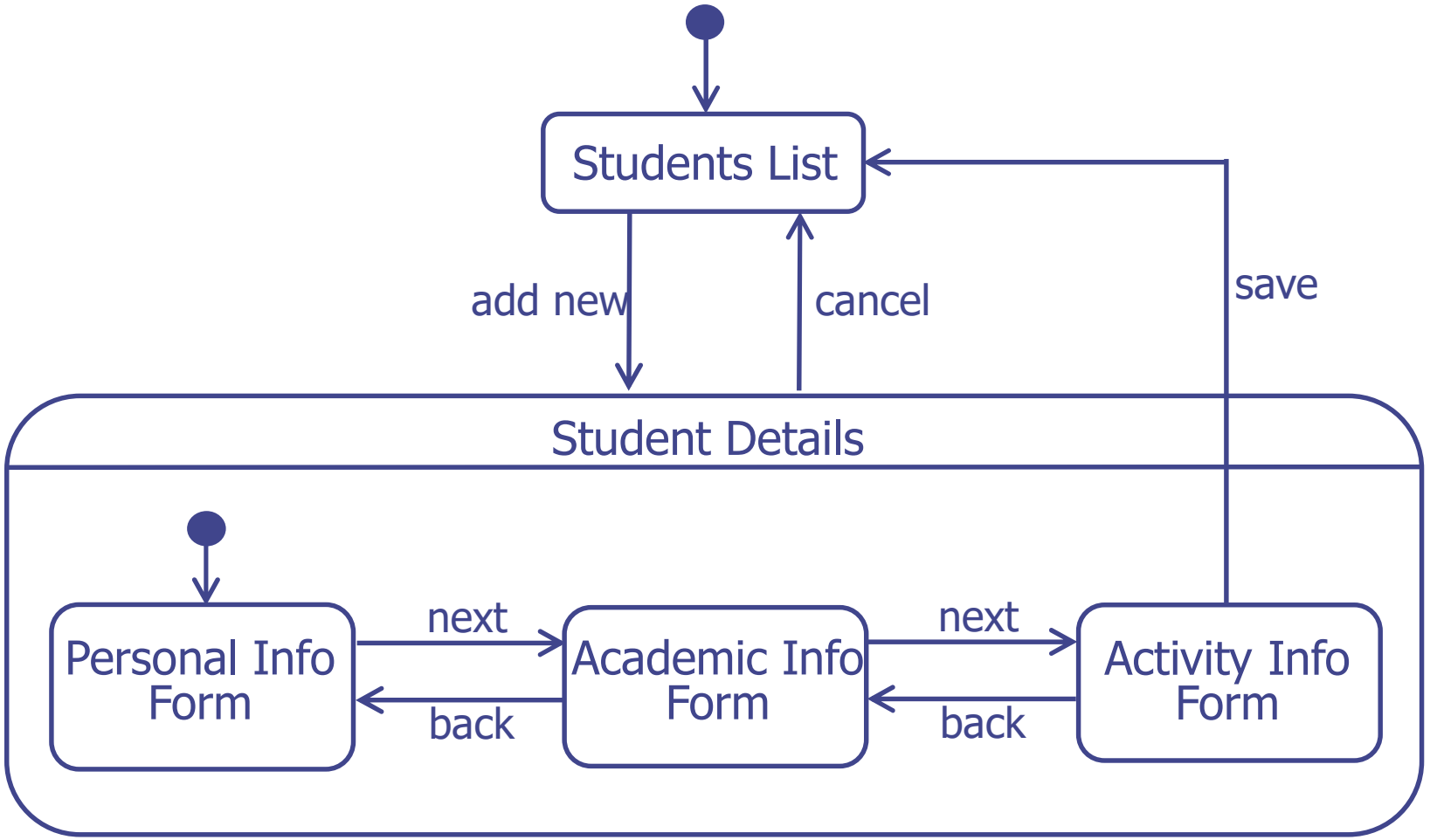
Such diagrams are sometimes called **dialog maps**.

- The dialog map helps us **find missing or incorrect navigation pathways early**, when they are cheap to fix.
- We can also spot opportunities for reuse and redundancies in the user interface, without worrying about the details of screen design prematurely.

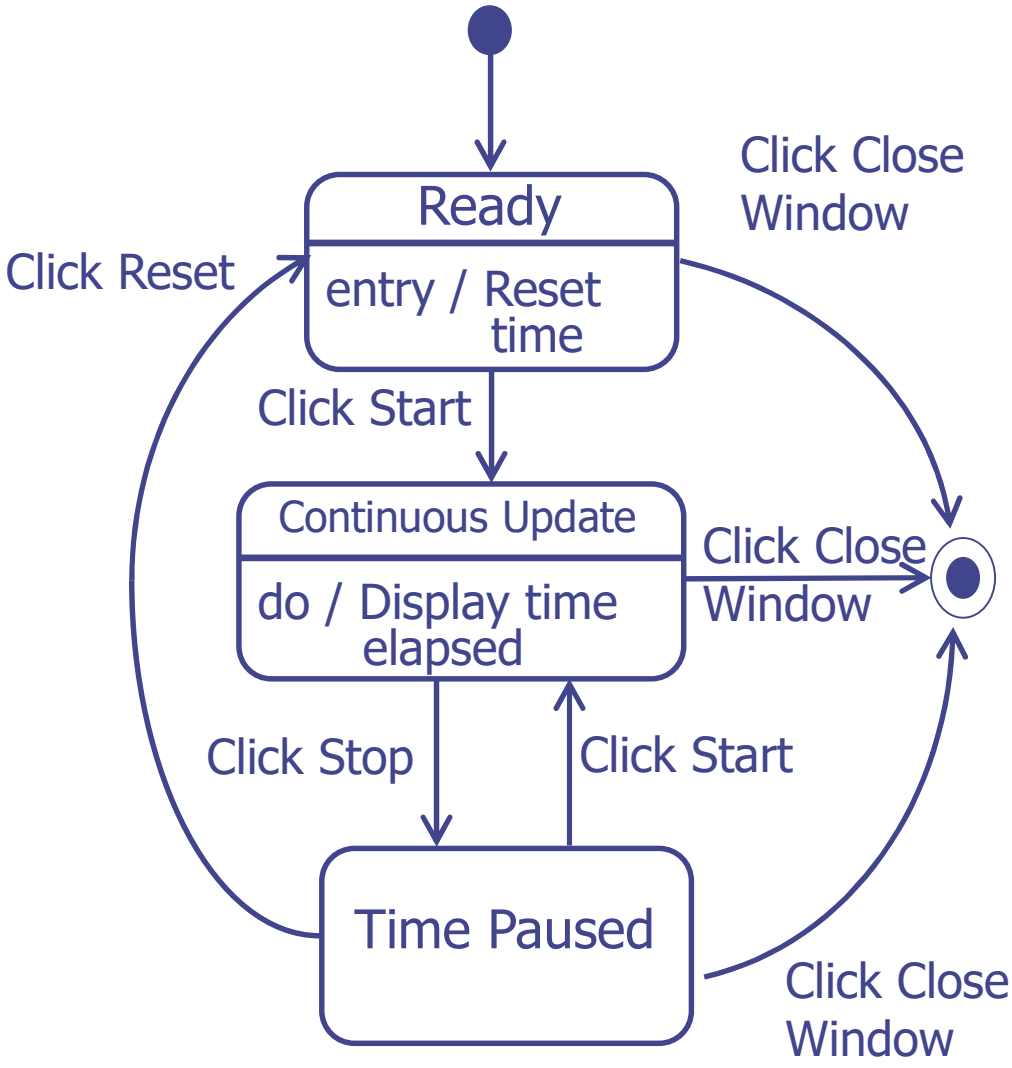
Example: Dialog Map of Vending Machine

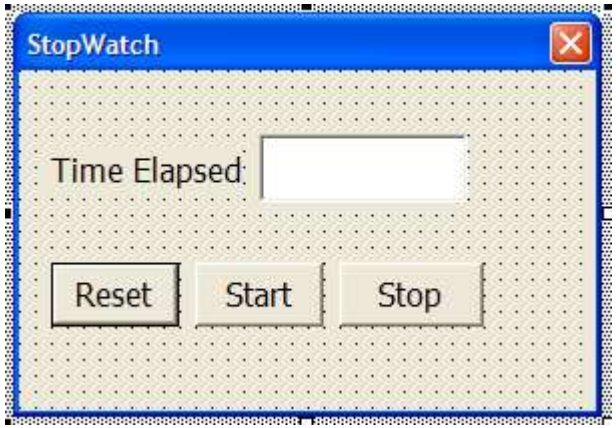


Example: Dialog Map of Student Management System



Practice: Stopwatch Functionality





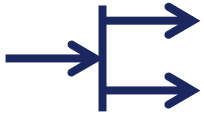
UML Activity Diagram



- Activities are the execution of one or more basic operations. Represented by a rounded rectangle.



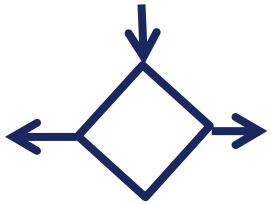
- Control flow between activities represented by arrows.



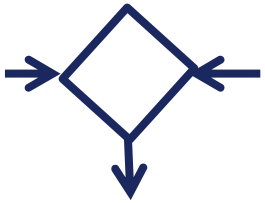
- Forks are when control flow has one input and two or more outputs (go to concurrent activities).



- Joins are when two or more control flows come together into a single control flow.

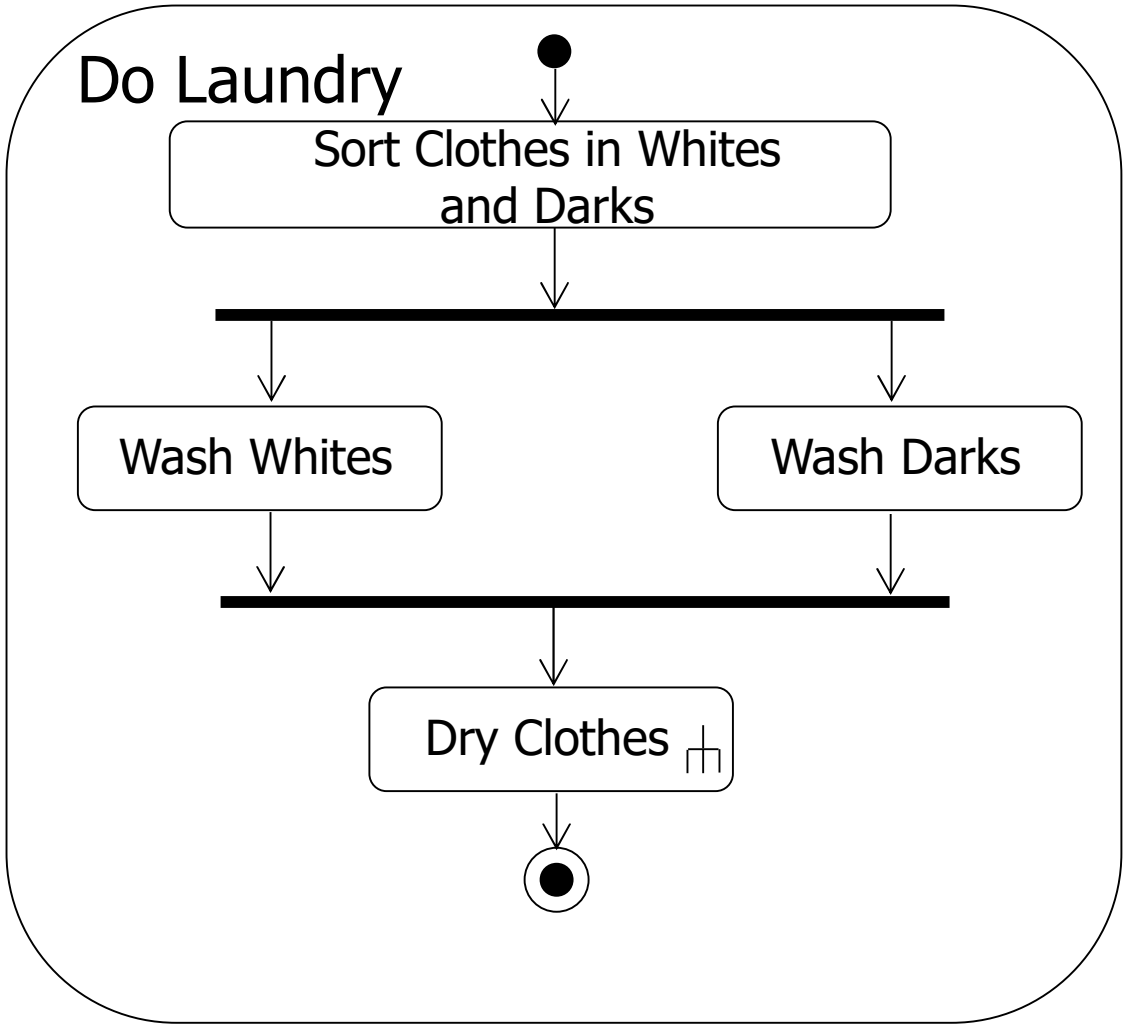


- Decision boxes (branches) are when control flow can go in one of several directions depending on a condition.

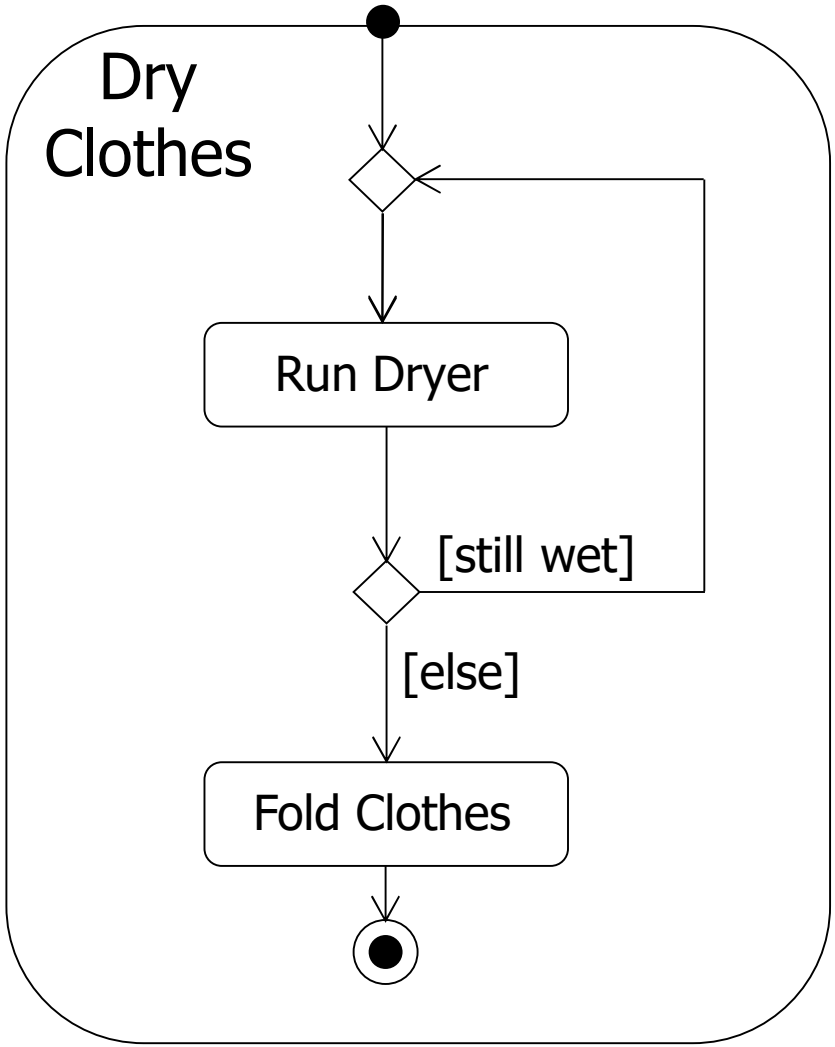


- Merge several branches into one branch

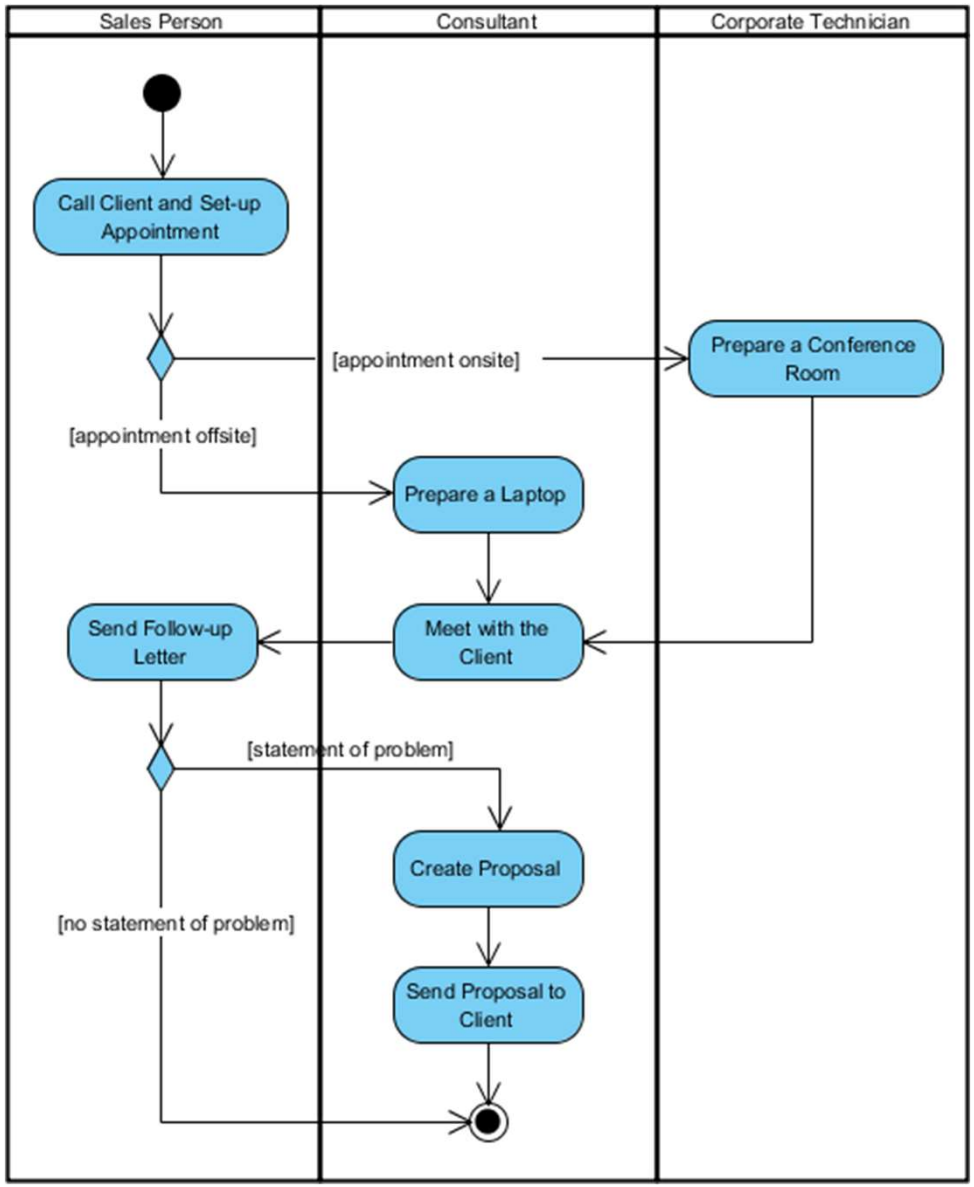
Example:
Activity Diagram of Do Laundry



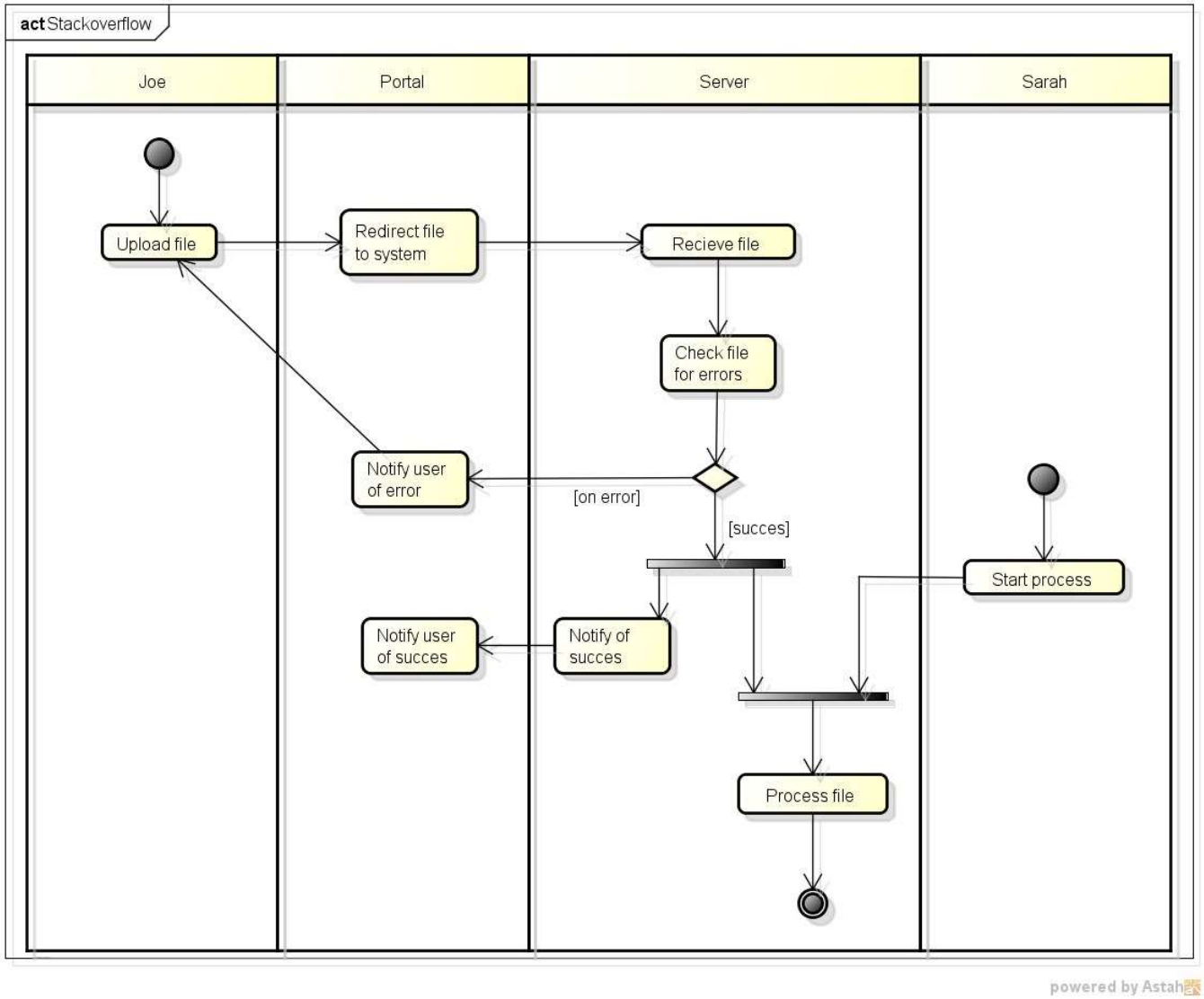
Example:
Activity Diagram of Dry Clothes



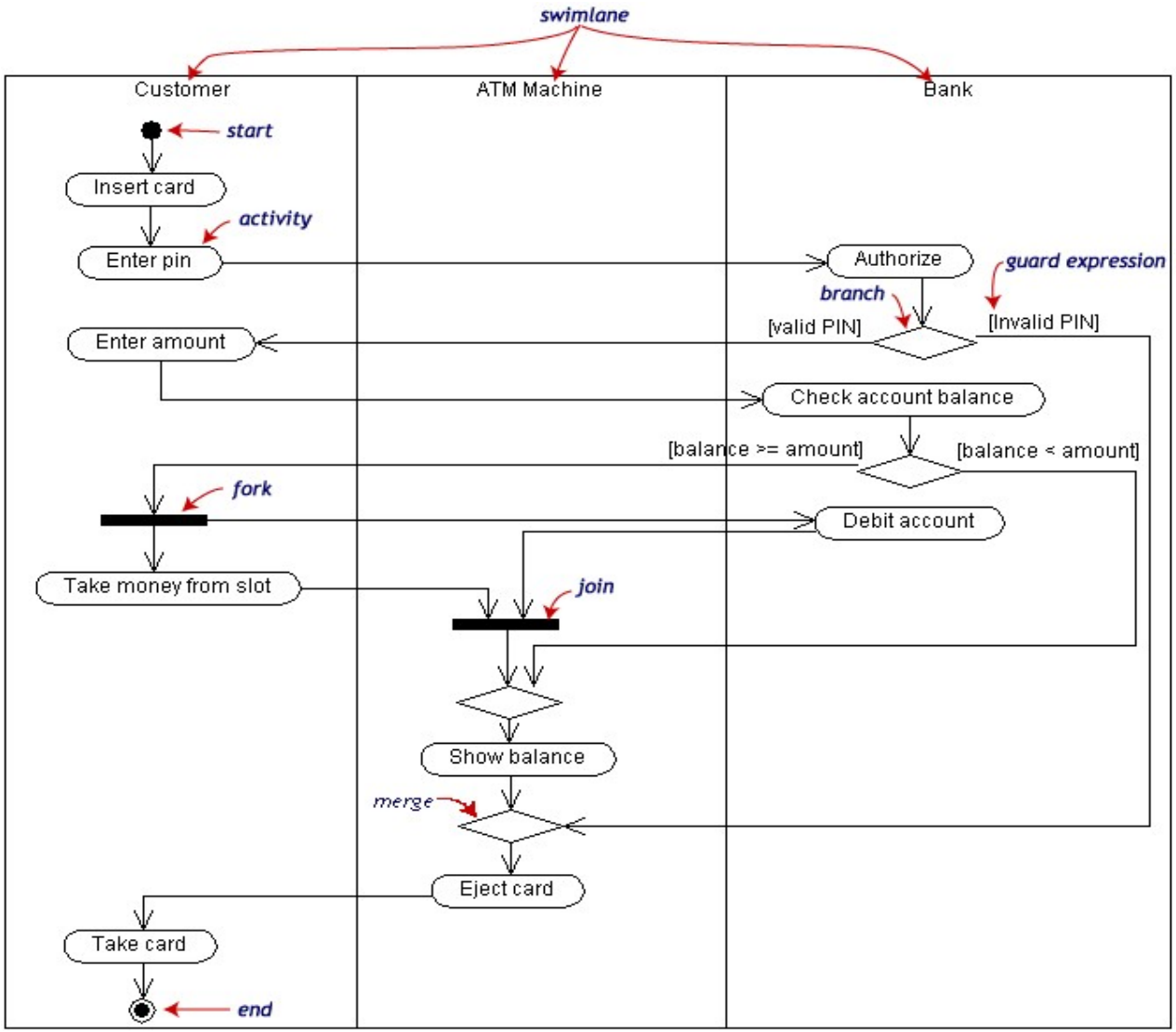
More Examples



More Examples



Example of Activity Diagram

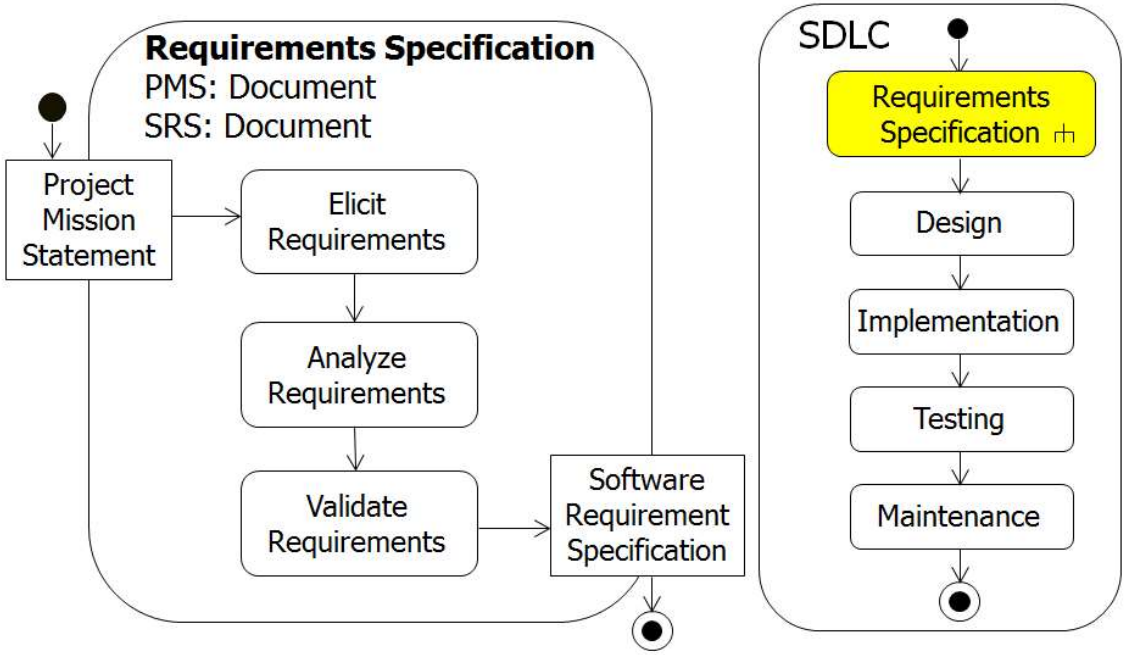


Difference between State Machine Diagram and Activity Diagram

Activity diagrams are used as a flow chart of **activities** performed by the system. They describe the **workflow behavior** of a system in terms of activities.

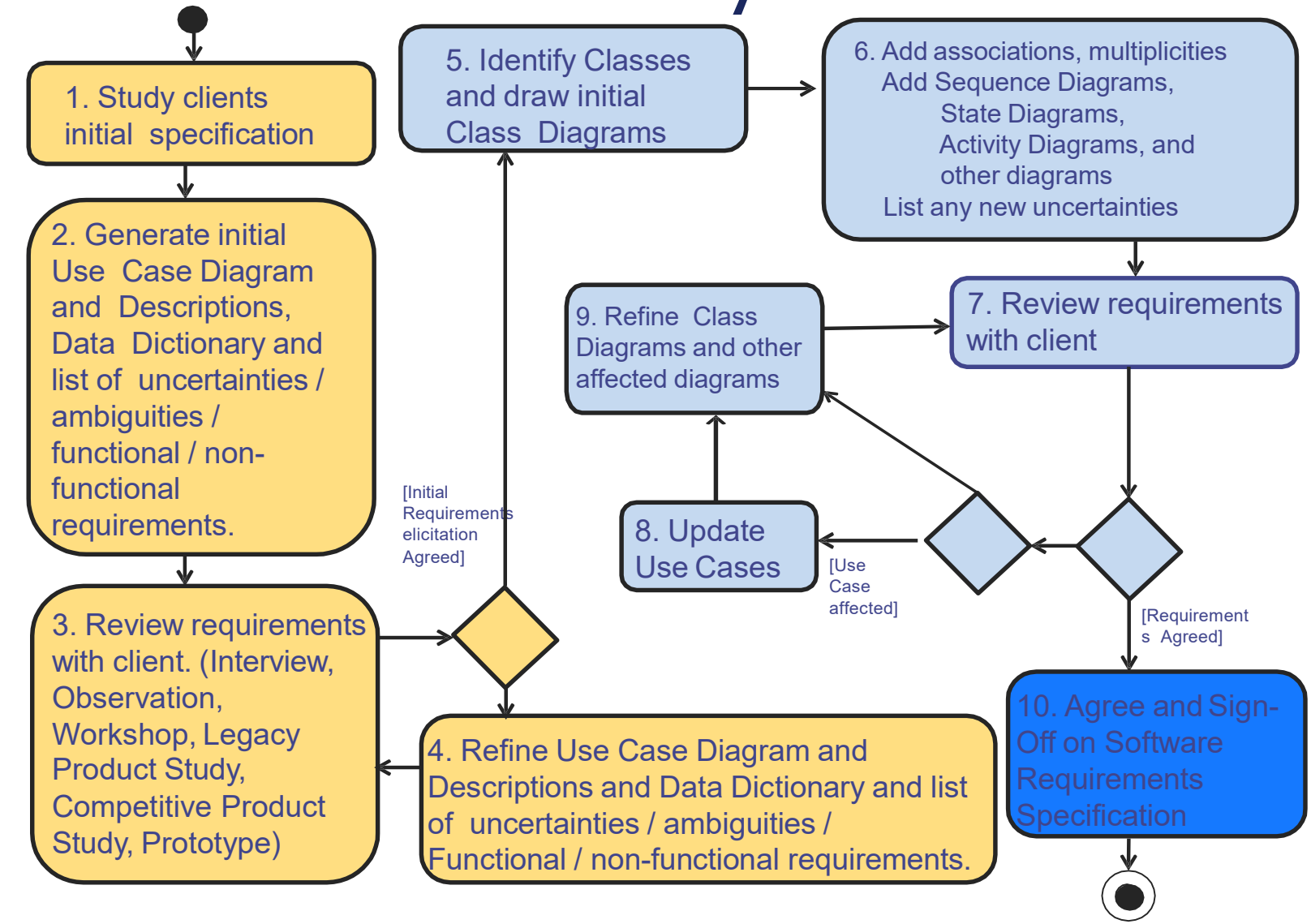
State machine diagram models **a system** or **an object** based on the "**States**" it can be in.

We have already used Activity Diagrams

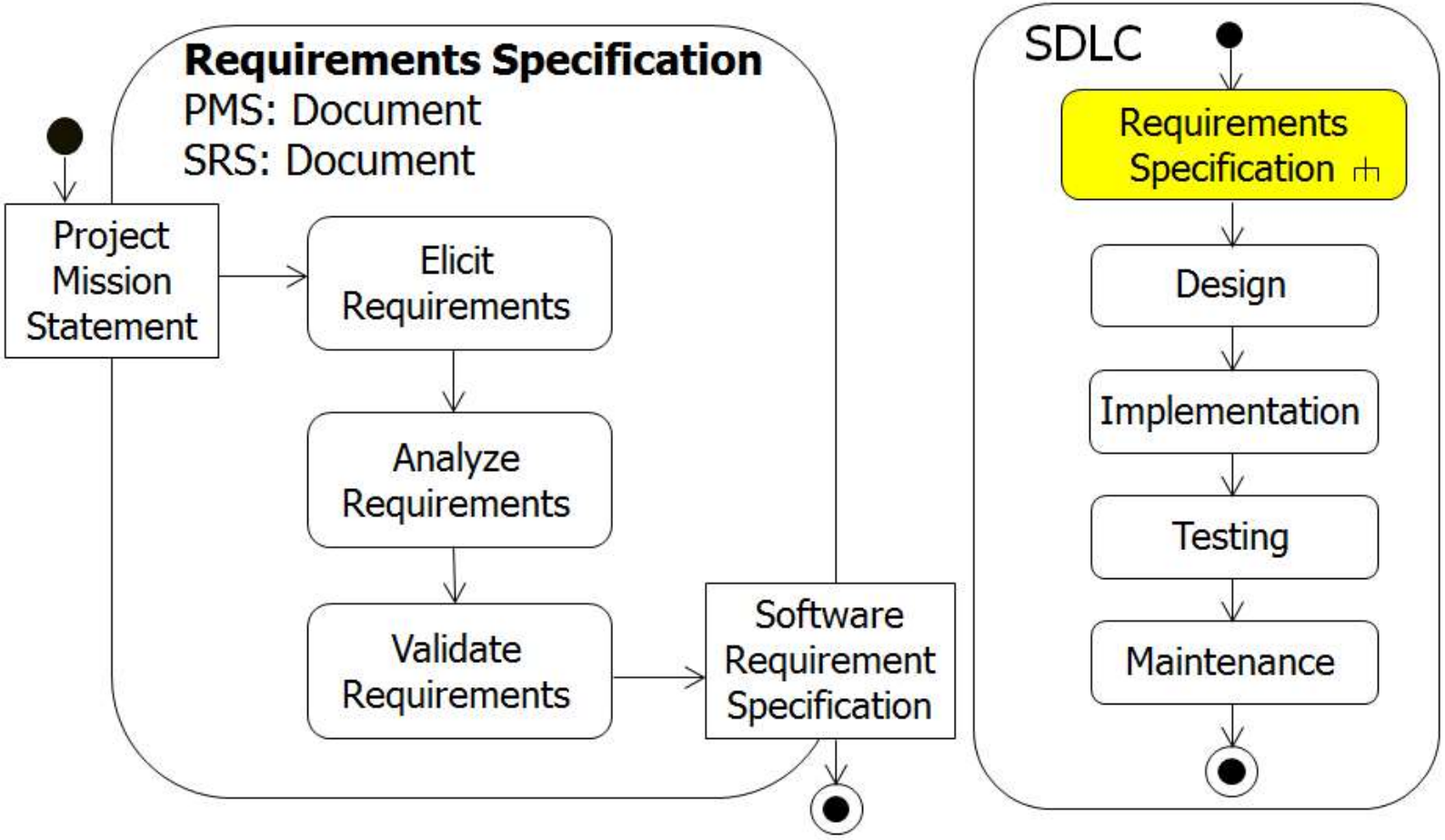


Review of Requirement Elicitation and Analysis Process

Activity Diagram of Requirement Elicitation and Analysis



Software Requirement Specification (SRS)



Contents of the SRS

1. Each organisation will have its own template for an SRS.

2. There are key content that must be addressed in an SRS

3. There is an IEEE standard
   830-1998 - IEEE Recommended Practice for Software Requirements Specifications.

   This is a document that you can download from IEEEXplore.

How and Who should write the SRS?

1. It should be written in **natural language**, in an **unambiguous** manner that may also include diagrams as necessary.

2. At the same time the detailed functional and non-functional **requirements need to be defined and understood** by the development team.

3. **Development team** members (programmers and managers) are **seldom the best writers of formal documents.** However, they often have to write it!

4. It is good practice to involve a **technical writer** with the development team in the drafting of the SRS because they are usually better at assessing and planning documentation projects and better meet customer document needs.

Contents of the SRS

1) **Product Description**
   a) **Purpose of the System (mission statement)**
   b) **Scope of the System**
   c) **Users and Stakeholders**
   d) **Assumptions and Constraints**
2) **Functional Requirements**
3) **Non-Functional Requirements**
4) **Interface Requirements**
   a) **User**
   b) **Hardware**
   c) **Software**
5) **Data Dictionary**

See detailed SRS description and SRS template on NTULearn course website.

Summary

- Interaction Diagrams
  - State Machine Diagrams
  - Activity Diagrams
- Software Requirement Specifications